

Knowledge-Augmented Conversational Agents for Dependency-Aware SysML v2 Model Generation and Refinement

Rui Wu

*Department of networked intelligence
Peng Cheng Laboratory
Shenzhen, China
wur01@pcl.ac.cn*

Guangwei Wen*

*Department of networked intelligence
Peng Cheng Laboratory
Shenzhen, China
wengw@pcl.ac.cn*

XiaoNing Zhang

*Department of networked intelligence
Peng Cheng Laboratory
Shenzhen, China
zhangxn@pcl.ac.cn*

Xiaojun Liang

*Department of networked intelligence
Peng Cheng Laboratory
Shenzhen, China
liangxj@pcl.ac.cn*

Chunhua Yang

*The School of Automation
Central South University
Changsha, China
ychh@csu.edu.cn*

Weihua Gui

*The School of Automation
Central South University
Changsha, China
gwh@csu.edu.cn*

Abstract—Model-Based Systems Engineering (MBSE) is pivotal for managing the complexity of modern systems. However, applying MBSE to complex products, especially in automatically generating accurate and consistent models in languages such as the Systems Modeling Language version 2 (SysML v2) from requirements, remains a significant challenge, primarily because of the novelty of SysML v2 itself, which is not well-represented in the training corpora of Large Language Models (LLMs), leading to inconsistent or incomplete designs. This paper introduces SysForge, a novel framework that addresses these challenges by transforming the LLM from a simple code generator into a collaborative design partner. Our framework introduces three primary contributions: a structured, comment-enriched sysml knowledge base enabling dependency-aware retrieval-augmented generation; an intermediate structured context synthesis step where an LLM agent refines retrieved information into a coherent summary; and an interactive Designer-Validator conversational loop that ensures correctness and integrity. We present an experiment to evaluate our framework against established baselines, demonstrating its potential to significantly improve the accuracy, completeness, and coherence of automatically generated SysML v2 models.

Keywords—SysML v2, Large Language Models, Retrieval-Augmented Generation

I. INTRODUCTION

Model-Based Systems Engineering (MBSE) has become an indispensable discipline for developing complex products across diverse sectors, including aerospace, automotive, and defense. By leveraging formal models to represent all aspects of a system throughout its lifecycle, MBSE aims to manage complexity, enhance communication, and ensure system integrity. The Systems Modeling Language (SysML), particu-

larly its latest iteration, SysML v2, provides a standardized syntax for this purpose. However, the practical application of MBSE is often hampered by the significant cognitive load and specialized expertise required for manual model development.

The advent of Large Language Models (LLMs) presents a transformative opportunity for automating and augmenting engineering design processes. Early explorations have demonstrated the potential of LLMs to generate code, including SysML, directly from natural language prompts. For instance, [1] highlights the use of LLMs as a direct, conversational interface for SysML v2, thereby simplifying user interaction and lowering the barrier to entry by abstracting away the complexities of traditional application programming interfaces. This trend signifies a critical shift towards more natural, human-like engagement with design models, ultimately democratizing access to powerful MBSE tools.

Despite these innovations, current approaches reveal inherent limitations when applying LLMs simply to the structured domain of systems modeling. This problem is exacerbated by the novelty of SysML v2. As a recently developed language, it is not sufficiently represented in the training corpora of most general-purpose LLMs, frequently leading to syntactic errors and a failure to accurately grasp language-specific semantics. This inherent knowledge gap renders LLMs unreliable for generating correct and consistent models. Furthermore, a standard Retrieval-Augmented Generation (RAG) approach, as proposed by Yu et al. [2], while improving accuracy by retrieving relevant examples from a knowledge base, still struggles with generating correct parametric views and often necessitates post-generation checks to enforce consistency.

The underlying issue is that both simple LLM prompting and basic RAG frameworks lack a deep, structural understanding of the system's architecture. An LLM might correctly

*Corresponding Author

This work supported in part by Guangdong S&T Programme under Grant 2024B0101010003 and the Major Key Project of PCL under Grant PCL2023A09.

generate a part definition based on a semantically similar example but fail to define its required port connections or link it accurately to other system components. The root cause lies in the inadequacy of semantic similarity between isolated text or code chunks to capture the intricate relational structure of a complete system model. To achieve reliable automation in MBSE, the paradigm must evolve from generating "text that looks like code" to creating "semantically and structurally valid models." This necessitates that artificial intelligence systems can access and reason over a structured representation of the entire system, rather than merely a flat collection of examples [10].

To address this fundamental gap, we introduce SysForge, a framework built on three contributions:

- 1) **Comment-Enriched SysML Knowledge Base (SysML-KB):** To endow the LLM with a holistic, structured understanding of the system, we move beyond simple vector-based retrieval to a structured RAG approach. The SysML-KB is built from a codebase where SysML v2 models are intentionally enriched with detailed, natural-language comments to enhance retrieval accuracy.
- 2) **Structured Context Synthesis:** To reduce noise and improve the quality of the initial generation, we introduce an intermediate step where a dedicated LLM agent synthesizes the retrieved, and often fragmented, code snippets into a coherent, structured natural language summary before passing them to the main generation agent.
- 3) **Designer-Validator Conversational Loop:** To proactively ensure model integrity, we propose an interactive, multi-agent refinement process that emulates an expert design review. A "Designer" agent generates the initial model, which a "Validator" agent then critiques against systems engineering principles and the SysML-KB.

This paper is structured as follows: Section 2 reviews related work in generative AI for systems engineering and advanced RAG techniques. Section 3 provides a detailed technical description of the SysForge framework. Section 4 outlines the experiment to evaluate our approach. Finally, Section 5 concludes the paper with a discussion of implications and future research directions.

II. RELATED WORKS

A. Generative AI in Systems Engineering

LLMs are being explored as collaborative partners across the entire engineering lifecycle, from requirements elicitation and analysis to system design and documentation [9]. DeHart et al. [1] leveraged LLMs to create a direct, conversational interaction layer with SysML v2 models. By allowing engineers to use natural language, this approach bypassed the need for intricate API knowledge, thereby lowering the technical barrier and democratizing MBSE. This work exemplifies the trend toward making complex engineering tools more intuitive and accessible. However, its focus remained on simplifying the user interface, with a reactive error-correction mechanism that depended on external tool feedback (e.g., compiler errors)

rather than an intrinsic understanding of design principles. Our work builds on this conversational paradigm but shifts the focus from interface simplification to ensuring the semantic and structural integrity of the generated model itself. Furthermore, a survey of benchmarks for CodeLLMs and agents reveals that the software design phase receives minimal attention (only 3%), highlighting a significant gap that SysForge aims to address [7]. Other research also explores GenAI for simulation model generation in MBSE, demonstrating LLMs' capabilities in generating system code and relationships [9].

B. Retrieval-Augmented Generation for Code

Retrieval-Augmented Generation (RAG) has become a standard technique for grounding LLM outputs in factual or domain-specific knowledge. In the context of MBSE, Yu et al. [2] implemented a RAG system that used multimodal vector embeddings to retrieve similar design examples, improving the accuracy of generated SysML models compared to a non-RAG baseline. However, the standard RAG paradigm has a critical limitation when applied to complex codebases: it fails to adequately handle code dependencies. A study by Gu et al. on repository-level code generation revealed that while retrieving contextual code from the current file and relevant APIs is highly beneficial, retrieving semantically similar but functionally unrelated code snippets can introduce noise and significantly degrade generation performance [3]. For instance, a request to model a "water pump" might cause a semantic RAG system to retrieve examples of "fuel pumps" and "oil pumps." While semantically close, these examples may contain irrelevant attributes (e.g., octane rating) or incompatible interface definitions, leading the LLM to generate an incorrect or incomplete model. The system lacks the structural context to discern which dependencies are relevant. This is particularly true for SysML v2, where the novelty of the language means that semantic similarity must be guided by more than just the code syntax itself, such as through well-written comments. An empirical study on RAG for code generation further confirms that noisy retrieved code snippets can decrease model performance [8]. Advanced frameworks like CodeRAG have begun to address this by explicitly modeling code dependencies, representing the state-of-the-art in dependency-aware code generation [4]. More recent work also explores knowledge graph-based approaches for repository-level code generation, emphasizing the tracking of inter-file modular dependencies and ensuring consistency with existing codebases [10].

III. THE SYSFORGE FRAMEWORK

Our proposed SysForge framework is designed to overcome the limitations of existing approaches by integrating a dependency-aware retrieval mechanism with an interactive refinement loop. This section details the architecture and core components of the framework.

A. System Architecture

As shown in Fig. 1, the framework operates through a multi-stage process orchestrated by a conversational agent:

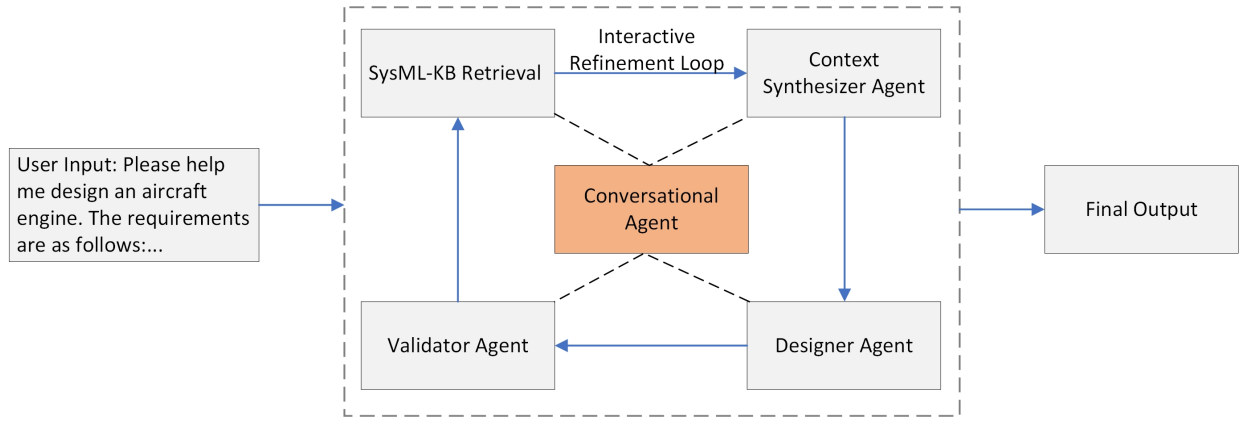


Fig. 1. The System Architecture of SysForge.

- 1) User input: The process begins with a user providing a high-level design requirement in natural language.
- 2) Conversational agent: This central component manages the interaction, parses the user's intent, and formulates a structured query for the SysML Knowledge Base.
- 3) SysML-KB retrieval: The query is executed against the SysML-KB. This is a hybrid process that first identifies relevant "seed" elements via semantic search and then queries the knowledge base to retrieve a complete, contextual set of related elements, including their critical dependencies.
- 4) Context synthesizer agent: An LLM agent receives the raw, retrieved code snippets. It synthesizes this information into a coherent, structured natural language summary that highlights key elements, relationships, and patterns for the next agent.
- 5) Designer agent: An LLM, prompted to act as a "Designer," receives the original user prompt along with the synthesized context. It then generates the initial SysML v2 code for the model.
- 6) Validator agent: A separate LLM instance, prompted to act as an expert "Validator," receives the generated code and the same synthesized context. It performs a structured review based on a predefined checklist of systems engineering principles and model integrity rules.
- 7) Interactive refinement loop: The Validator's feedback is passed back to the Designer agent, which uses this feedback to refine the code. This loop continues until the Validator approves the design.
- 8) Final output: Once validated, the complete and coherent SysML v2 code is presented to the user.

B. Core Components and Workflow

The SysForge framework is composed of three primary stages, each corresponding to a key innovation.

1) *Knowledge graph construction and retrieval*: The foundation of our framework is the SysML Knowledge Graph (SysML-KG), a high-fidelity representation of a SysML v2 codebase's architecture. The KG is constructed through an automated pipeline that performs deep structural analysis on the source code. This process moves beyond simple text

parsing to systematically deconstruct the models, identifying not only core entities (e.g., 'PartDef', 'PortDef') but also, critically, their semantic and structural relationships (e.g., composition, connection, and inheritance). The result is a rich, queryable graph where nodes represent model elements and edges represent their dependencies. This 'design-as-a-graph' paradigm is the key to enabling dependency-aware generation.

A key aspect of our methodology is the enrichment of the source code with detailed, natural-language comments. For each node i in the knowledge graph, its textual content T_i is a concatenation of its descriptive comment D_i and its raw code snippet C_i . This content is encoded into a vector embedding E_i by a code-specialized language model:

$$E_i = \text{Encoder}(\text{concat}(D_i, C_i)) \quad (1)$$

This ensures the embedding captures both semantic meaning and structural information. When a user submits a query Q , it is vectorized to V_q . The Retrieval Agent performs a hybrid search. First, it identifies a set of seed nodes S_{seed} by finding the top-k most similar entries in the KG:

$$S_{\text{seed}} = \arg \max_{i \in \text{KG}} \left(\frac{V_q \cdot E_i}{\|V_q\| \|E_i\|} \right) \quad (2)$$

Second, it performs contextual expansion by traversing the KG from the seed nodes in S_{seed} to retrieve all directly connected nodes and their relationships, forming the complete retrieved context $C_{\text{retrieved}}$.

2) *Structured context synthesis*: Raw retrieved context $C_{\text{retrieved}}$ can be noisy or redundant. Our second innovation, the Context Synthesizer Agent (A_{synth}), addresses this by pre-processing the context. This agent is an LLM prompted to act as a "Technical Analyst," tasked with transforming the collection of code snippets into a clean, structured summary C_{synth} :

$$C_{\text{synth}} = A_{\text{synth}}(C_{\text{retrieved}}) \quad (3)$$

This synthesized context, which might summarize key parts, common attributes, and observed connections, is then passed to the Designer Agent. This step ensures the Designer receives a clear, distilled set of design patterns rather than having to infer them from fragmented code.

3) *Iterative refinement and validation*: Our third innovation is the multi-agent refinement loop. The Designer Agent (A_{design}) receives the user query Q and the synthesized context C_{synth} to generate an initial model, M_0 :

$$M_0 = A_{design}(\text{format}(Q, C_{synth})) \quad (4)$$

This model is then passed to the Validator Agent (A_{valid}), which acts as an expert reviewer. At each iteration j , the Validator assesses the model M_j against a predefined checklist (covering syntactic validity, connection integrity, etc.) and produces structured feedback F_j . If there are no issues, $F_j = \emptyset$ and the loop terminates. Otherwise, the feedback is returned to the Designer Agent, which generates a revised model M_{j+1} :

$$M_{j+1} = A_{design}(\text{format}(Q, C_{synth}), M_j, F_j) \quad (5)$$

This iterative process, where M_{j+1} is a function of the original prompt, the previous model state, and the validator's critique, continues until the model is approved ($F_j = \emptyset$) or a maximum number of iterations is reached.

IV. EXPERIMENTS

A. Experimental Setup

The knowledge base for our experiment is constructed from a specialized dataset comprising 526 SysML v2 code samples. This dataset includes 249 official examples sourced from the SysML v2 GitHub release repository [5] and 277 custom-developed code samples designed to cover a range of modeling scenarios. This combined dataset is partitioned for knowledge base construction and a held-out set of test prompts. The experimental data is divided into two parts: the data used to build the knowledge base, which consists of 426 entries, and the data used for testing, which comprises 100 entries. Each test dataset is consisted of a natural language prompt and a ground-truth SysML v2 model for comparison. All generative agents within the SysForge framework and the baseline models utilize deepseek-v3 [6] as the foundational large language model.

In addition, we use Pass@k and BLEU to provide evaluation. Pass@k is the metric that evaluates functional correctness by measuring the percentage of problems for which at least one of the top k generated code samples passes a set of unit tests. For our experiments, we use Pass@1, which measures the rate at which the single generated output is correct. The general unbiased estimator for Pass@k is given by:

$$\text{Pass@k} = \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (6)$$

where n is the total number of generated samples per problem, c is the number of correct samples, and k is the number of samples considered. BLEU measures the quality of the generated code by comparing the overlap of n-grams (contiguous sequences of n items) between the generated code and the reference code. It is calculated as the geometric mean of

TABLE I
THE EXPERIMENTAL RESULTS

Method	Pass@1(%)	BLEU
DeepSeek-V3	0.0	0.23
Semantic RAG (Baseline)	42.0	0.45
SysForge	95.0	0.75

the modified n-gram precisions (p_n), multiplied by a brevity penalty (BP) to penalize generated outputs that are too short:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (7)$$

where w_n are positive weights summing to 1, and N is typically 4.

B. Results

The primary results of our comparative evaluation are summarized in Table 1. The analysis of these results reveals a clear progression in performance, underscoring the effectiveness of each architectural enhancement in the SysForge framework. The standard LLM approach, lacking any external context, performed poorly, achieving a Pass@1 rate of only 0.0% and a BLEU score of 0.23. This confirms that modern LLMs struggle to generate correct SysML v2 code due to its novelty and domain-specific syntax. The introduction of a standard RAG methodology provided a substantial improvement, with the Semantic RAG baseline increasing the Pass@1 rate to 42.0% and the BLEU score to 0.45. This result demonstrates that providing relevant examples is crucial for grounding the model and improving basic correctness. However, the most significant performance gain was achieved by the complete SysForge framework. It far surpassed both baselines, reaching a Pass@1 of 95.0% and a BLEU score of 0.75. This result demonstrates that a more sophisticated, multi-agent approach is necessary to achieve high-fidelity model generation. The combination of dependency-aware retrieval, context synthesis, and the iterative Designer-Validator loop allows SysForge to not only select better context but also to proactively identify and correct errors, leading to a final output that is both syntactically correct and semantically aligned with the user's intent.

Fig. 2 presents a partial visualization of the SysML v2 code generated by SysForge. This visualization is produced using the official SysML v2 compiler, which concurrently serves as an implicit validation step: incorrect or inconsistent code would result in compilation errors, thereby reinforcing the integrity and correctness of SysForge's outputs.

V. CONCLUSION AND FUTURE WORK

This paper introduced SysForge, a novel framework for generating SysML v2 models from natural language. We have argued that the primary challenge for LLMs in this domain is not a lack of language fluency but a lack of structural understanding. Our work addresses this gap with three key contributions: a comment-enriched SysML Knowledge Base

Part of the sysml v2 code visualization diagram

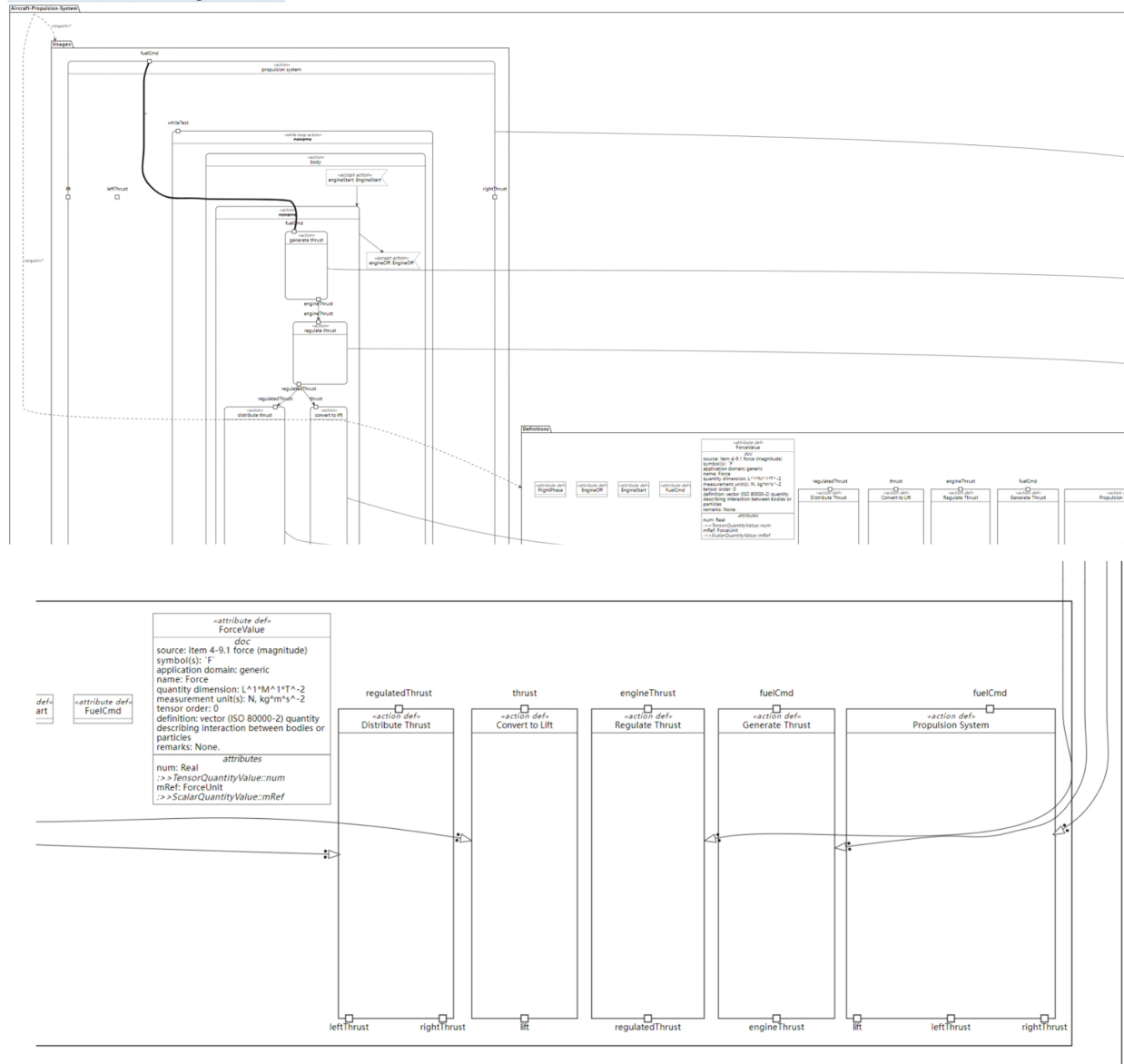


Fig. 2. Part of the sysml v2 code visualization diagram. If the generated code is incorrect, an error will be reported during code visualization.

(SysML-KB) that enables dependency-aware retrieval, a context synthesis agent to refine retrieved information, and an interactive Designer-Validator loop that ensures model integrity. By transforming the LLM from a simple text generator into a context-aware design partner, our framework represents a significant step toward reliable and automated MBSE.

The implications of this approach extend beyond performance improvement. By making the AI's "knowledge" of the system explicit in a structured knowledge base and its "reasoning" process transparent through the structured validation dialogue, we enhance both the capability and the interpretability of the system. This is particularly crucial for building trust and enabling adoption in safety-critical design

domains where black-box solutions are unacceptable.

Despite its promise, our framework has limitations. Future work will proceed along several exciting avenues:

- 1) Numerical and Parametric Reasoning: A known weakness of current methods is the generation of parametric diagrams. We plan to integrate our framework with external tools.
- 2) Multi-Modal Knowledge Base: The current SysML-KB is limited to code. We will extend it to become a multi-modal design repository by incorporating diagram images, requirements documents, and simulation results.
- 3) Scaling and Enterprise Integration: We will investigate advanced database indexing and retrieval techniques to

ensure the framework can scale to massive, enterprise-level model repositories.

REFERENCES

- [1] J. K. DeHart, "Leveraging large language models for direct interaction with sysml v2," in INCOSE International Symposium, vol. 34, no. 1. Wiley Online Library, 2024, pp. 2168-2185.
- [2] H. Yu, Z. Chen, X. Xiong, Y. Dai, and H. Cai, "Intelligentmbse design approach based on retrieval augmented large languagemodel," *Journal of Graphics*, vol. 45, no.6, 2024, pp. 1188.
- [3] W. Gu, J. Chen, Y. Wang, T. Jiang, X. Li, M. Liu, X. Liu, Y. Ma, and Z. Zheng, "What to retrieve for effective retrieval-augmented code generation? An empirical study and beyond," *arXiv preprint arXiv:2503.20589*, 2025.
- [4] J. Li, X. Shi, K. Zhang, L. Li, G. Li, Z. Tao, F. Liu, C. Tao, and Z. Jin, "Coderag: Supportive code retrieval on bigraph for real-world code generation," *arXiv preprint arXiv:2504.10046*, 2025.
- [5] M. Bajaj, S. Friedenthal, and E. Seidewitz, "Systems modeling language(sysml v2) support for digital engineering," *Insight*, vol.25, no.1, pp. 19-24, 2022.
- [6] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan et al., "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [7] K. Wang, T. Li, X. Zhang, C. Wang, W. Sun, Y. Liu, and B. Shi, "Software development life cycle perspective: a survey of benchmarks for code large language models and agents," *arXiv preprint arXiv:2505.05283*, 2025.
- [8] Z. Yang, S. Chen, C. Gao, Z. Li, X. Hu, K. Liu, and X. Xia, "An Empirical study of retrieval-augmented code generation: challenges and opportunities," *arXiv preprint arXiv:2501.13742*, 2025.
- [9] L. Zhang, Y. Zhang, Y. Zhang, D. Niyato, J. Sun, B. Zhang, H. Yu, and J. Wu, "GenAI for simulation model in model-based systems engineering," *arXiv preprint arXiv:2503.06422*, 2025.
- [10] M. Athale and P. J. K. M., "Knowledge graph based repository-level code generation," *arXiv preprint arXiv:2505.14394*, 2025.